7a.PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTICES 100
struct Node {
int vertex;
struct Node* next;
};
struct Graph {
struct Node* adjLists[MAX_VERTICES];
int visited[MAX_VERTICES];
};
struct Queue {
int items[MAX_VERTICES];
int front;
int rear;
};
// Create a node
struct Node* createNode(int v) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->vertex = v;
newNode->next = NULL;
return newNode;
}

// Create a graph
struct Graph* createGraph() {
struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
int i;
for (i = 0; i < MAX_VERTICES; i++) {
graph->adjLists[i] = NULL;
graph->visited[i] = 0;
}
return graph;
}
// Add edge
void addEdge(struct Graph* graph, int src, int dest) {
// Add edge from src to dest
struct Node* newNode = createNode(dest);
newNode->next = graph->adjLists[src];
graph->adjLists[src] = newNode;
// Add edge from dest to src
newNode = createNode(src);
newNode->next = graph->adjLists[dest];
graph->adjLists[dest] = newNode;
}
```

```
// Create a queue
struct Queue* createQueue() {
struct Queue* queue = (struct Queue*)malloc(sizeof(struct Queue));
queue->front = -1;
queue->rear = -1;
return queue;
}
// Check if the queue is empty
int isEmpty(struct Queue* queue) {
return queue->rear == -1;
}
// Add an item to the queue
void enqueue(struct Queue* queue, int value) {
if (queue->rear == MAX_VERTICES - 1)
printf("Queue is full\n");
else {
if (queue->front == -1)
queue->front = 0;
queue->rear++;
queue->items[queue->rear] = value;
}
}
// Remove an item from the queue
int dequeue(struct Queue* queue) {
int item;
if (isEmpty(queue)) {
printf("Queue is empty\n");
item = -1;
} else {
item = queue->items[queue->front];
queue->front++;
if (queue->front > queue->rear) {
queue->front = queue->rear = -1;
}
}
return item;
}
// Print the graph
void printGraph(struct Graph* graph) {
int v;
for (v = 0; v < MAX_VERTICES; v++) {
struct Node* temp = graph->adjLists[v];
printf("\n Adjacency list of vertex %d\n ", v);
while (temp) {
printf("%d -> ", temp->vertex);
temp = temp->next;
```

```
}
printf("\n");
}
}
// BFS traversal
void BFS(struct Graph* graph, int startVertex) {
struct Queue* queue = createQueue();
graph->visited[startVertex] = 1;
enqueue(queue, startVertex);
while (!isEmpty(queue)) {
int currentVertex = dequeue(queue);
printf("Visited %d\n", currentVertex);
struct Node* temp = graph->adjLists[currentVertex];
while (temp) {
int adjVertex = temp->vertex;
if (graph->visited[adjVertex] == 0) {
graph->visited[adjVertex] = 1;
enqueue(queue, adjVertex);
}
temp = temp->next;
}
}
}
int main() {
struct Graph* graph = createGraph();
addEdge(graph, 0, 1);
addEdge(graph, 0, 2);
addEdge(graph, 1, 2);
addEdge(graph, 1, 3);
addEdge(graph, 2, 3);
printf("Graph:\n");
printGraph(graph);
printf("\nBFS Traversal starting from vertex 0:\n");
BFS(graph, 0);
return 0;
}
```

OUTPUT:

Graph:

Adjacency list of vertex 0
1 -> 2 ->

Adjacency list of vertex 1
0 -> 2 -> 3 ->

Adjacency list of vertex 2
0 -> 1 -> 3 ->

Adjacency list of vertex 3
1 -> 2 ->

BFS Traversal starting from vertex 0:
Visited 0
Visited 1
Visited 2
Visited 3